

Challenges and industry practices for managing software variability in small and medium sized enterprises

Tuomas Ihme · Minna Pikkarainen · Susanna Teppola ·
Jukka Kääriäinen · Olivier Biot

Published online: 24 April 2013

© Springer Science+Business Media New York 2013

Abstract Software variability is an ability to change (configure, customize, extend) software artefacts (e.g. code, product, domain requirements, models, design, documentation, test cases) for a specific context. Optimized variability management can lead a software company to 1) shorter development lead time, 2) improved customer and improved user satisfaction, 3) reduced complexity of product management (more variability, same \$) and 4) reduced costs (same variability, less \$). However, it is not easy for software companies, especially small and medium size of enterprises to deal with variability. In this paper we present variability challenges and used practices collected from five SMEs. Our study indicates that increased product complexity can lead growing SMEs to the time-consuming decision-making. Many of the analyzed medium size of companies also expect improved tool support to help them to boost their productivity when managing increasingly complex products and increasing amount of variants. In fact, in many of the analysed SMEs, a high level of automation in design, release management and testing are or become a key factor for market success. By introducing the challenges and used practices related to

Communicated by: Dieter Rombach

T. Ihme · M. Pikkarainen (✉) · S. Teppola · J. Kääriäinen
VTT Technical Research Centre of Finland, P.O. BOX 1100, 90571 Oulu, Finland
e-mail: minna.pikkarainen@vtt.fi

T. Ihme
e-mail: tuomas.ihme@vtt.fi

S. Teppola
e-mail: susanna.teppola@vtt.fi

J. Kääriäinen
e-mail: jukka.kaariainen@vtt.fi

O. Biot
Sirris, The Collective Centre of the Belgian Technological Industry, Diamant Building,
Boulevard A. Reyerslaan 80, 1030 Brussels, Belgium
e-mail: olivier.biot@sirris.be

variability the paper deepens understanding of this highly relevant but relatively under-researched phenomenon and contributes to the literature on software product line engineering.

Keywords Software variability challenges · Variability practices

1 Introduction

Nowadays, the trend in software companies is that decisions impacting product and service functionality are made at more late stages of the development (Bosch 2006; Hallsteinsen et al. 2008; Jaejoon et al. 2012). Another trend in software companies is to build customized products for multiple domains (Bosch 2006), which had led many software industries to a portfolio of increasingly diverging and complex legacy products (Metha and Heineman 2002). In this case the business strategy of companies is often to give users or customers a high variance of product and service features to be used and configured for the specific purposes.

Variability in software products can be characterized as an ability to change, configure, customize or extend software artefacts for a specific context. (Bachmann and Clements 2005; Svahnberg et al. 2005). The products and services that the software companies aim to build often differs from each others supporting individual customers or different market segments (Chen and Ali Babar 2010). For example a television set product family can introduce variability for different customer segments supported by the multimedia standards and peripheral interfaces.

Variability management deals with managing variation in both time and space dimensions (Ali Babar et al. 2010; Krueger 2001). The “*variation in time*” concept (Bosch et al. 2001; Pohl et al. 2005) means that software products and artefacts evolve over time. In the *space* dimension, software is used and varied in multiple contexts at the same time (Bosch et al. 2001; Halmans and Pohl 2003). Kratochvíl and Carson (2005) have found that even half of companies producing software (i.e. software company) face variability challenges.

An SME (Small and Medium Enterprise) is defined as a company that has less than 250 employees (European definition¹) or less than 500 employees (US definition²).³ Variability management has some special characteristics in SMEs. In the context of customer-driven industrial systems engineering, SMEs often deliver customized products with accompanying services like installation or user training (Stallinger et al. 2011). Typically SMEs have very limited resources to allocate for the increasing product variability (Thörn 2010). Due to the resource situation SMEs tend to use less time for requirement engineering (Groves et al. 2000), structuring and modelling (Thörn 2010). As a consequence, documentation and current practices of SMEs can be unsuitable for variability management (Verlage and Kiesgen 2005). For instance, SMEs typically use ad hoc types of practices to implement reuse (Thörn 2010).

Variability management in software companies is extremely complex and challenging tasks that even in SMEs need to be supported by different methods, techniques and tools. Although there has been a great deal of research addressing challenges of variability management, there has been only little effort to systematically analyse what are the actual

¹ http://ec.europa.eu/enterprise/policies/sme/facts-figures-analysis/sme-definition/index_en.htm

² <http://stats.oecd.org/glossary/detail.asp?ID=3123>

³ In this paper we are using US definition of the SMEs.

variability challenges in small and medium sized software enterprises. The purpose of this multiple case study with semi-structured interviews is to identify and analyse various challenges and used practices that the SMEs are currently facing on the way to optimizing variability management. Specifically, we pursue this objective through the following research questions:

- 1) what are the variability challenges in SMEs at the moment
- 2) what practices are the SMEs currently using to manage software variability?

Although research provides several discussions about variability (Reinhartz-Berger and Sturm 2009, Bosch 2006; Gulp et al. 2001; Ali Babar et al. 2010), small and medium sized enterprises still lack guiding practices to solve their major variability issues (e.g., Chen and Ali Babar 2010). Furthermore, it remains unanswered in their research how the SMEs can benefit from variability (Thörn 2010). Understanding the variability challenges and the used practices helps the research community to better understand the concept of variability. For SME, the description of their variability challenges and used practices in different contexts, help them to achieve successful variability management strategy and to optimize management of product variants in their own domain.

This paper is structured as follows. Section 2 presents the background for this study including a discussion about the different variability challenges that are currently reported in literature. Section 3 introduces the research approach used in this study. Section 4 presents variability challenges and used practices that we found in five SMEs companies. Section 5 discusses about the findings. Section 6 concludes the paper, provides limitations and some ideas for the future research activities related to this topic.

2 Background

This section describes the software variability practices and challenges that form the background of this research.

2.1 Variability Permeates Different Aspect of Software Development

In business to business companies, product managers are continuously evaluating change requests coming from different customers, originating from end users (Patton 2002). Change impact needs to be defined from both technical and business perspective (Clements and Northrop 2001; Info-Tech 2004). Decision making about the variability of product and service artifacts needs to be supported similarly as in any decision making combining traditional financial measurements with supplementary measures and real time information of the product or service status (Info-Tech 2004).

During software development, products and services go through a number of development phases. Each development phase has its own artifacts such as a user input, technology expectations, requirement specification, architecture description, design documentation, source code, compiled code, linked code, running code and test and maintenance documentation (Gulp et al. 2001). For example, in the testing, companies need to 1) set up a strategy how to test different product and service artifacts 2) determine which artifacts are affected by changes and 3) ensure the consistency of configurations. Thus, the variability needs to be modeled as a part of the test case scenarios (Patton 2002).

2.2 Software Variability Challenges

Both Chen and Ali Babar (2010) and Bosch et al. (2001) state that it is difficult for companies (both SMEs and large companies) to handle variability and variability models in requirements engineering stage where the company can get huge amount of visualized features and high diversity of different stakeholders. In these situations, it is also difficult to assess and understand implications of the changes to different requirements or features on the software system (Bosch et al. 2001). The evolution of variability often requires evolving product or service architecture (Chen and Ali Babar 2010). However, software companies often lack of architectures that directly support continuous variability management (Chen and Ali Babar 2010). Optimized variability management requires special software engineering knowledge to identify common non-functional characteristics and varying functional features (Galster and Avgeriou 2011). The functions and non-functional characteristics need to be identified componentizing the existing code and its interfaces that, according to Ali Babar, are often missing in the software companies.

Development of products is economically constrained and variability can be seen as a key approach in developing systems or software with fewer resources. According to Asikainen et al. (2007), “products that incorporate variability can address multiple user segments, allow price categorisation, support various hardware platforms and operating systems, provide different sets of features for different needs, and cover different market areas with different languages, legislation, and market structure”.

Instead of creating a single product or service, companies are rather building either a family of similar products and services or a large complex product portfolio and platforms that are used in order to satisfy the continuous stream of needs originating from different stakeholders. According to Hallsteinsen et al. (2008), product line engineering “provides a means of customizing variants of mass-produced products”. Mass customization can be defined as “producing goods and services to meet individual customers’ needs with near mass production efficiency” (Hallsteinsen et al. 2008). It is obvious that in this situation, each variability decision must be thoroughly considered. Making good decisions is however difficult and requires both a considerable amount of information and systematic working practices which are often difficult for companies to arrange. For example, architects and management do not often have enough knowledge of the cost, resource implications as well as dependencies between the architectural elements to make successful variability decisions (Bosch et al. 2001).

The systematic use of reusable assets and their variants using software product line practices has brought significant benefits (van der Linden et al. 2007). A software product line adoption in a very small organisation (25 developers) has produced many experienced improvements such as reduction of time to market by 2–4, reduction of maintenance costs by 60 %, reduced cost of quality (reliability in the field) and the break-even point of up-front investments was reached after about five products (van der Linden et al. 2007). The Software Engineering Institute (Northrop 2010) has reported that achieved advantages include as much as a ten-fold increase in quality, decreased cost by as much as 60 %, decreased labour needs by as much as 87 %, decreased time to market by as much as 98 % and ability to move into new markets in months, not years. The above experienced improvements cohere with two critical success factors in the long-term survival of over 1,600 small manufacturing enterprises analysed by Kim et al. (2008). One of them is the ability of the firms to produce quality products with a longer product life cycle at a competitive price. Another one is the use of the firms’ product and management experiences to make their products more appealing to the customer.

In addition to many success stories, lesson from software product line adoption includes also several challenges. For example, the risks, timetables, efforts, and costs experienced from software product line adoptions can be a barrier for the product line approach adoption for many organisations (Krueger 2002). Stallinger et al. (2011) identify seven key challenges a medium-sized software organisation is facing in software product line adoption: process diversity in the face of multiple reuse approaches; the management of requirements and variability; the integration of requirements traceability and variability management; legacy software and discipline vs. software-specific modularization; integration with systems engineering; costing and pricing models; and project vs. product documentation.

Some empirical research emphasizing the challenges of variability is outlined in Table 1. Initial categories of the variability dimensions were derived from the literature, which was based on the real life, industrial situations. The created challenge categories (common themes) are used as a basis for the inductive data analysis described in Section 4.

As shown in Table 1, software product line research has provided a multiplicity of variability management approaches, discussions and solutions to variability related challenges in companies for nearly 20 years (Reinhartz-Berger and Sturm 2009; Bosch 2006; Gorp et al. 2001; Ali Babar et al. 2010). However, an important issue is that the output of variability management research has reported to be largely mismatched with industrial needs (Chen and Ali Babar 2010; Ali Babar et al. 2010). Less than 30 % of the proposed approaches have been evaluated in an industrial setting (Chen and Ali Babar 2011). A software company might need to evaluate the suitability of a variability management approach for the specific domain and needs of the company (Ali Babar et al. 2010). Variability management research and practice have mainly focused on proactive-oriented variability design and binding variation points before delivering the product. Modern business ecosystems demand a higher degree of variability and adaptability from their products (Hallsteinsen et al. 2008). In the context of modern business ecosystems, a mixed variation point binding approach might be more viable, where some variation points related to the product's core structures and properties are bound before delivery, some others are bound by partners and distributors, additional variation points are bound when software is launched to adapt to the current use environment and the last bindings are done during operation to adapt to changes in the environment (Hallsteinsen et al. 2008).

Variability management is different depending on the context of product development (Codenie et al. 2010; Thörn 2010). For example, Thörn (2010) argues that SME consultants often have no significant reusable assets because they can be obligated to handover the results of their work to the customer. It can also be different depending on the size of the software enterprise. For example, Thörn (2010) introduces a survey of the current state and potential variability management practices in 260 SMEs focusing on the organizations that have less than 50 employees. The survey summarizes that there is more awareness of the potential benefits of a structured variability management especially when the organization grows.

We found few reports (Fillis 2001; Mtigwe 2005; Laporte et al. 2008; Thörn 2010, Groves et al. 2000, Kamsties et al. 1998; Nikula et al. 2000; Mohagheghi and Conradi 2007; Rech et al. 2007; Manolova et al. 2002) about software variability practices and challenges especially in small and very small organizations. At first we found interesting findings from Thörn's (2010) survey. We gathered additional findings from the rest of the articles for the more detailed cases of this study. We compiled the following interesting and comparable findings from previous work regarding reuse challenges and observations in small and very small organisations:

Table 1 Variability Challenges in software companies

Challenge category	Galster and Avgeriou 2011	Chen and Ali Babar 2010	Bosch et al. 2001	Thörn 2010
Technical variability challenges				
Impact analysis		Difficulties to understand implications of different features on customers' buy-in	Difficulty to assess the impact of changes due to the different meaning to concepts by different stakeholders Software architects lack an awareness of the cost information.	
Product status management	Difficulties to identify varying functional feature based on the requirements description	Extracting the variability and commonality of existing system artefacts Management of evolving requirements of SPL	Dependencies between architectural elements and features are seldom made explicit. Visibility to the variability at the requirements and realization level	Difficulties in managing the increased size of product variants and several concurrent software versions
Decision making		Difficulties in managing architectural design decisions	Wrong decisions of the earliest binding time	
Reuse		Requirements are not reused, everything is build from scratch in ad hoc manner		Communication gaps and duplicated work
Change implementation (architecture level)	Difficulties to identify common non-functional characteristics for the product	Componentizing the existing code and building variability inside and around them	Difficulties to trust into the variants that are added by the 3th party after construction of the product	
	Translation of variation points into variability model	The existing architecture does not support the required variability in the new requirements	Difficulties to find all code that belongs to one feature	
		Difficulties extracting the variability from technical artefacts of different similar products	Design is often oversimplified by architects	
		That builds a common architecture		
		Difficulties to compare code bases		

Table 1 (continued)

Challenge category	Galster and Avgeriou 2011	Chen and Ali Babar 2010	Bosch et al. 2001	Thörn 2010
Testing	Difficulties to identify valid variation points	Large amounts of efforts are spent on software testing in industry	A large amount of variability may make it virtually impossible to test all combinations during development.	
Tool support		Complex management of variability models during requirements engineering	Lack of sufficient tool support to automate variability management	Lack of tools to do systematic reuse
		Lack of sufficient tool support for managing variability		
		Variability modelling approaches are not very user friendly		
People variability challenges				
Resourcing		Lack of good architects who are good at thinking at an abstract level, who know the mechanisms to make the architecture extensible and flexible, and can apply those principles in practice	It is difficult to predict how the variant will impact to the resource-usage during design.	
		Reuse is too time consuming		
Business models		Existing business model does not encourage reuse		

- *Lack of resources and budget to describe reuse assets and reuse processes* (Laporte et al. 2008, Thörn 2010)
- *There is a need for better documenting reusable assets.* In the SMEs evaluated by Rech et al. (2007) the documented information about reusable assets was only partially complete, consistent, dependable, or valid. In addition, it was difficult to locate and share.
- *Little requirements or design reuse* (Thörn 2010; Groves et al. 2000; Kamsties et al. 1998)
- *Little traceability between the problem and solution spaces* (Thörn 2010)
- *Small organisations are good at designing solutions for reuse* (Thörn 2010)
- *The observed benefits of systematic industrial reuse.* The observed benefits of systematic industrial reuse in small products are lower problem density, decreased correction efforts and increased productivity (Mohagheghi and Conradi 2007; Thörn 2010).
- *Proper product variability structures are seen useful.* According to Thörn (2010), SMEs see that proper variability structures for product, problem and solution spaces increase productivity, improve communication and testing, help configuration and version management, simplify maintenance and reduce costs, bugs, and work duplication

In addition, we compiled the following interesting and comparable findings from previous work regarding variability management practices in small and very small organisations:

- *Competitive advantage via flexible products and services.* The ability to offer flexible, customised products or services provides the differentiation that enables very small firms to compete and gain competitive advantages over larger firms in certain market niches (Fillis 2001; Mtigwe 2005).
- *Competitive advantage via specialized experts and face-to-face communication between them.* Very small organisations tend to compensate for reuse challenges and informal management practices by specialized expertise and face-to-face communication, decision-making and problem resolution (Laporte et al. 2008; Mtigwe 2005; Manolova et al. 2002).
- *Late requirement changes.* Customer satisfaction is often determined by the fulfilment of specific requirements that may change during the project (Laporte et al. 2008).
- *Flexible and informal managerial practices.* Very small organisations tend to have flexible and informal managerial practices (Laporte et al. 2008; Mtigwe 2005).
- *The number of supported product variants is usually low* (Nikula et al. 2000; Thörn 2010).
- *Customers are systems integrators.* Very small organisations are usually focused on developing custom systems, where the customer is in charge of the management of the whole system and software integration, installation and operation (Laporte et al. 2008).

3 Research Design

3.1 Research Method

A multiple case study with semi-structured interviews was selected as main research method for this study due to several reasons. First of all, a multiple case study is a good way to get more in-depth understanding of the current industrial variability challenges and solutions. Secondly a case study is best in supporting the how and why questions (Yin 2002), hence it

serves well our purpose to identify the variability challenges, and used practices through different industry domains in SMEs.

3.2 The Case Companies

The case selection in this study was based on theoretical sampling (Eisenhardt and Graebner 2007), the purpose being to find cases that would be particularly suitable for shedding light on the phenomenon of variability challenges and used practices. The companies were carefully selected for this study based on their need for variability management and their size. All of the selected have a common need to manage a large amount of product variants and the increasing complexity of their product or a service portfolio. The following criteria were used in the case selection: 1) software or software & hardware companies, 2) complex product development, 3) existing product/service portfolio, and 4) large amount of stakeholders. Background (i.e. type and size of the analysed companies) is described in Table 2.

- Company 1 The first organization we studied is an SME with 38 people developing scheduling and dispatching software systems for the needs of passenger and freight transport. The company had offices in three countries, however, the great majority of R&D was located in one country. The company focus has been to make different solutions for the two markets, therefore the products of the company were based on two product families. In the field of product variability management the main intention of the company was to support better reusability *via* simplifying the code base towards better modularity. Also targets were settled to support better reuse between the two product families, so that new features could be better used in the both existing product families. The software development in one of the product families is based on the Scrum approach with two weeks' Sprints. Requirements and team members tend to change too much during development iterations in the other product family.
- Company 2 is a small research and development centre of a SME corporation that provides custom solutions and services for mobile phones, personal computers and electronic products. The company has around 400 full time staff members working for them in global context. The interviewee's respective development centre in Finland is responsible for one specific software- and hardware-based product type that has about 150 own customers. Some years ago the decision

Table 2 Background of the analysed companies

Company	Domain of the company	Size of the company
1	Transport systems	Tens of employees/less than ten in the product team interviewed
2	Mobile Devices,	Hundreds of employees/less than ten in the development centre interviewed
3	Automotive	Hundreds of employees/13 in the unit interviewed
4	Medical, Space and banking industry	Tens of employees
5	Meteorology and the environmental science	Hundreds of employees

was made in the company to emphasize to the configuration management techniques. The partnership with the other key players in the field together with the efficient use of customized development and configuration approach is key for their success in the dynamic markets.

- Company 3 is developing control systems for industrial vehicles and machines. The product domain is in the area of critical environments where demanding safety and trust related features are important. The company has about 200 employees in two countries. The project base was formed so that half of the engineers were working with the company's own products and the rest were working directly for the customers. The person that was interviewed in this study represented a small software development unit (13 employees) which main responsibility was on developing a Linux platform for two different HW platforms (altogether the company had approximately five different product platforms). Product variants and other realizations were then built on the top of these platforms. Mostly product variants were created in a way that several customers could be served by it, but in some cases also customer specific product variants were made for a single customer. The main intentions in the area of product variability were to decrease the complexity of the products by increasing on component based development, and by improving reusability in different levels of design.
- Company 4 is a small company that provides design services to different industrial domains such as banking, space and media industry. The strategy of the company is to provide project based subcontracted hardware and software-based design solutions to larger system integrators, using a large variety of different technologies.
- Company 5 is an SME having few hundreds of people making systems for meteorology and the environmental science. The company is making customized products, out of the box products and services. Their strategy is to reuse platforms to make products and services for their customers. Their market is evolving relatively slowly and because of the safety reasons the customers want to use the same products for a long period of time and without many changes. This is causing so-called endless projects in which the maintenance period is really long. These projects should be managed in parallel to new product development.

3.3 Data Collection and Analysis

The role of the researchers in all cases was to understand the company's variability challenges and used industry practices. The data collection for this study was done through a series of case studies where observations, regular meetings, 15 semi-structured interviews and various workshops were used as a data collection method in five companies. All together, the data collection involved 10 persons, representing a research project coordinator, site managers, technology managers and project managers from the Finnish and the Belgian industry. Most of the data from the 60–90-minute interviews were tape-recorded and transcribed. Researchers took extensive notes. In those cases, in which the recording was not possible, the analysis was based on the notes of the researchers.

The results were produced using inductive and deductive analysis (Patton 2002), adopting within-case and cross-case analysis approaches as described by Eisenhardt (1989). The analysis process in this study consisted of two phases, namely 1) within-case

analysis (Eisenhardt 1989), in which the strengths and barriers of variability were identified, and 2) cross-case analysis (Eisenhardt 1989), in which the results were compared among the different cases. In the first phase, each of the five cases was analysed in a deductive manner using tabulation (Runeson and Höst 2009) in which the variability challenges and used practices were added to the tables in order to see the overview of the situation. In the second phase, the findings from each case were produced using inductive analysis (Patton 2002), discovering common patterns and themes. Open coding was first used in the second phase in order to identify the variability management challenges employed in the case companies.

Information obtained from the interviews concerning external customer reference practices was confirmed by searching the company websites and company-specific marketing materials. The data from all companies studied were further analysed focusing on the reasons behind the problems that occurred in the case companies, and describing the case similarities and differences. The common patterns and themes, which emerged from each case, were compared with the practices produced during the second analysis phase. Categorization of the common patterns and themes were developed during the analysis every time when a new challenge or practice category emerged from the interview data. To demonstrate the consistency of the findings and the reasoning through which the data was interpreted, the findings are reported together with illustrative, representative quotations from the interview respondents.

3.4 Validation

The prolonged involvement strategy (Karlström and Runeson 2006) was used in two of the studied companies to reduce the following threats to validity in the data collection and study analysis: misconceptions, misunderstanding, omissions and misinterpretation of intent. The strategy provided the researcher with an excellent opportunity to gain insights into the required background. All research team members did not participate in this longitudinal collaboration. This was assumed to provide a good balance between the researcher bias threat and the respondent bias and reactivity threats (Karlström and Runeson 2006).

Multiple data sources (data triangulation) were used: observations, regular meetings, semi-structured interviews and workshops. Two or more researchers participated together in the semi-structured interviews and workshops (investigator triangulation). Interpretive, positivistic and critical research perspectives (theory triangulation) were represented in the interview and workshop sessions, data analysis and feedback (member checking) phases of the study. The interviews were recorded and transcribed professionally (audit trail).

This study has following limitations: our data was collected only from few individual people from the five companies. Hence the results are limited to the respondents' knowledge of his/her company situation about the software variability challenges and practices. To eliminate this limitation, we selected interviewees that have high position and therefore high impact to the variability in their companies. Sample size might be another issue as we had only 10 participants in our semi structured interviews that were conducted in five SMEs software organizations.

Despite the abovementioned and other potential limitations, reasonable countermeasures have been implemented in the study to reduce the threats to the validity of the study.

4 Research Results

Section 4 describes the identified variability management challenges and used industry practices deployed by the case companies.

4.1 Technical Variability Challenges and Industry Practices

4.1.1 Impact Analysis

Results show that handling product complexity cause challenges for analysed SMEs (C1, C4, C5). In three of the analysed companies, there is a need to combine knowledge of the customer needs and the prior knowledge of the product in order to understand the impact and feasibility of a modification. In the analysed cases, plenty of discussions and long negotiation periods are needed with customers before it becomes clear whether the variant is valuable to be build or not.

4.1.2 Status Management

Results indicate that in these change situations it is often challenging for SMEs to communicate about core assets and of the status of the implemented product variants. For example: in Company 5, information of the product status is not efficiently transferred from several Scrum projects to managers and the other stakeholders (e.g. sales and marketing). It was mentioned by a manager of company 5 that the product status information needs to be well documented when making new tenders for the customers as well as long term release and product planning.

Documentation is one way to manage status of large amount of product variants. However, large amount of time consuming documentation of product variants and core assets were reported as a variability challenge for three SMEs. For example in Company 3: *“it is a challenge to maintain documentation while the development changes are continuously happening.”* (C3). In these cases (e.g. C3 and C5), complexity and the amount of products in a portfolio can be a documentation management challenge. There is a challenge and a need to develop documentation practices for the used architectural style in Company 1.

Improved tool support for requirements, release and configuration management were mentioned as solutions for documentation problems in three SMEs (2, 3 and 5): *“a web based tool in which the customer can select which features to get in the next release would help us automate the release delivery process and to assure that the customers get what they ordered.”* Manager, C2.

4.1.3 Decision Making

It was found that most of the studied SMEs have a clear criteria to help decision making of upcoming product variants. In fact, in all of the companies analysed impacts of the upcoming requests were evaluated in the following levels: 1) how much time the development would take, 2) how much money the development will cost and 3) what else could we have for the same amount of money. However, results indicate that it is difficult for SMEs to evaluate the return of investment of their variability decisions. For instance, the managers of Company 5 stated that it is difficult to evaluate the return on investment of the appeared changes: *“it is not a criterion for us, how many customers are interested to buy the new changed features. It is more interesting if one customer is willing to pay a lot or pay more because of this one implementation”*. C5.

4.1.4 Change Implementation

Time consuming change implementation was a major consequence of product portfolio complexity in two SMEs (C1, C4). For example, a manager of Company 1 noted that *“Even those change requests that first seem to be very small, cause an enormous effort to*

implement in reality” (C1). In practice, change requests need to be implemented every time when a company is planning to sell something to customers. Three of the analysed companies (1, 2 and 5) use agile development conventions that allow late changes in requirements. Requirements tend to change also during development iterations in Company 2.

Some companies try to hide the product complexity in a sales presentation to a prospective customer. This happens in Company 1 since due to the complexity, it is always a problem to integrate the system with other (new and existing) technologies and software solutions. Company 1’s strategy is to make customisations by setting parameters without a need for changes to the basic product. In spite of that, customisations for some important customers can sometimes conflict with the above strategy and the needs of the whole product. However, hiding the real efforts from customers can be risky and lead the company later on to a difficult situation, in which they can not deliver on time or fulfil all the customer expectations.

A common practice for two SMEs (C1, C2) is to decrease the change implementation time using parameters and modularity. The parameters may be used in the situations in which the company has good, clear knowledge about the parameters that are needed or relevant for specific customers. According to Company 1’ manager: “Although competitors have the similar type of products, the main reason for the better results is that our service algorithms contain more varied functionality in variable usage situations.” Another example in Company 2: *“some of the parameters are identified based on rules or regulations, some of them are coming directly from the customers”*.

As the previous quotation shows, parameters are often identified based on the experience of the designers. For example Company 1’ experts make majority of customizations by setting parameter values of advanced service algorithms at the installation stage to meet the special needs of individual customers without a need for changes to the basic product. Company 2 delivers very complex and customer-specific product releases. It has been successful in competition because of the automated and fast configuration capacity of the company’s reuse platform. In Company 2: *“the configuration tool reads the code, modifies it and removes parts of it depending on which features we want to add in the release that will be delivered to a specific customer”* Company 2 applies a practice to ask the sales or marketing person to define in customer contracts what features must be delivered to their customers. The parameters of the configuration system are selected and set according to the needed customer features. The configuration tool makes the customer release based on the parameter list.

Selective product re-engineering was reported as another solution to decrease the time used for change implementation in Company 1. In analysed cases, it is not possible to do re-engineering because of time and resource constraints. In some cases it was suggested to be better to divide the re-engineering to smaller pieces, which makes it easier and cheaper to implement and manage. In addition, in the words of Company 2’s manager interviewed, “If the impact of changes from a customer is very big, we can just say sorry, but we can’t do that.”

4.1.5 Documentation

Document templates and patterns (e.g. reusing requirement descriptions, feature descriptions, interfaces and documentation in Company 4) were often used to decrease documentation time. In Company 2, customer deliveries include documentation that is generated from the version management system: “the generated document is not necessarily the best form

for documentation that should be continuously available for customers to select what features they want for the next releases.” Reuse of previous features was not reported as a problem for Company 1 because the company and its product are small enough that everyone knows which features have been built earlier.

Results suggest that software companies often have an intention to increase reuse of the platform components. For example, reuse is actively used in Company 5, in which the goal is to build a platform that could be reused as a part of the next upcoming product. Their challenge is that the company does not have documented requirements that could have been used while reusing components from the existing product platforms. The absence of proper requirement documentation makes reuse of product platform components a time consuming activity: *“Reuse takes much time because we must always go to the code level in order to understand what the component is all about” (C5).*

Results indicate that reusability can be an investment for SME especially in the cases in which the new functionality is included into product platform. Challenges occur if the manager who is making decisions about the reusable assets does not have a clear vision about the future customer needs and payback period. In one case (C2) *“Integrator customers have very good knowledge about the product’s domain and they are very careful on selecting which features they actually want to be reused in platform level.”* Manager, C2.

4.1.6 Testing

In testing situations, effect management was identified as a variability challenge in two of the analysed case companies (C2 and C4). According to Company 2’s manager, they have no time and capacity for testing sufficiently delivered releases. Thus more test automation is needed. Test automation helps also Company 5 to manage product variants. Company 2 does not use open source components due to defect, virus and other risks.

Another example from the manager of Company 4: *“When we do the change we have to manage the diversity of versions in the different levels [different phased in product development life cycle] is difficult”*. A similar problem was identified in Company 2 in which the manager argued *“How to manage defects and how to make updates to the customers in the maintenance phase. The problem is that one defect can be in a system bought by 150 customers”*. This is the case especially when every one of the customers has several versions of the same system.

To deal with this problem, Company 2 put the following actions in place. Firstly, they keep a history of the customers and the deployed releases at their premises. Secondly, detailed criteria are used to identify when to make changes due to a defect: *In the case of a fatal defect we always first check how many customers have this defect. Then we check how many of these customers have a valid maintenance contract. If the defect is not fatal, we evaluate if the defect is worth fixing.* Manager, C2. Thirdly, customers are allowed to modify the product (e.g., to integrate it with other systems) only via ‘wrappers’ that hide the product’s dependencies on its environment such as operating systems.

4.1.7 Tool Support

Code baseline, scripts and a release tool are used for generating release delivery packages to customers in Company 2. The package includes a definition of all parameters, configured code, documentation and other material that have been delivered to the customer. Every time when a customer release is made, the release package is tested and all package data is

automatically stored into the database of customer releases. As the manager from Company 2 suggested, at the moment *“it takes about one hour for an engineer in our company to change something in the system. In the past we made everything by hand and it took much longer.”*

Some of the interviewed managers made practical suggestions how to improve future complexity management practices in SMEs. One suggested solution made by company 2 was to *“automate release baseline implementation in a way that the releases would be automatically generated for the customers based on their feature identification in a cloud application.”*

4.2 Non-Technical Challenges

4.2.1 Resourcing

Three of the SMEs analysed (C2, C3, C5) reported that they do not have enough resources for upcoming product development and customization activities. In these conditions, it is often unclear for a company whether to focus on maintenance, new product development or platform development. For instance, in Company 5, developers had several “endless” maintenance projects on-going at the same time when new products are developed.

4.2.2 Business Models

Results show that the uncertainty of the business model in use has an impact on product variant pricing. For example, Company 2’s manager commented: *“there are cases in which technical support could yield more than selling the product. Sometimes the product is given for free and the money is coming from maintenance and technical support”*. It was found in some companies that the used pricing mechanisms do not always support the used variability management strategies. For instance, Company 2 has a plan to create web based tool in which the customer could easily select the product configuration that they need or are willing to use in their specific context. This approach would, however, require a change in the pricing mechanism that the company is using. Instead of giving different price of the overall product configuration to the different customers, the company should rather use catalogue type pricing mechanisms, in which the price is identified separately for each functionality or the overall product or a service.

Transition from one customer segment to another is every day life to the one of the companies analysed (C 5). In fact, Company 5 is having 5 different customer segments that all have different needs based on their domain specialties. In these situations, the product is becoming as a kind of tool box that the company is offering to their customers. Different customer segments easily cause a continuous stream of very different customer requirements, each of which can have a different meaning in different domains. Efficient customer collaboration is reported as a practice that needs to be improved especially in SMEs that are moving from one customer segment to another (C5). In this company, the customer collaboration methods and social media tools were recommended by the managers to assure continuous communication of the customer needs in different domains. Company 1’s both products aim to gain competitive advantage in a certain narrow market niche without tight price competition and among about twenty customers in terms of profitable product prices, better product quality, shorter time-to-market and low maintenance costs. For keeping

maintenance costs low Company 1 delivers the newest releases of the basic product to all customers.

4.3 Differences and Similarities with Previous Work

Table 3 summarizes interesting differences and similarities between the findings that were gathered from previous work in Section 2 and the results of this study regarding reuse challenges and observations in small and very small organisations.

Table 4 summarizes interesting differences and similarities between the findings that were gathered from previous work in Section 2 and the results of this study regarding variability management practices in small and very small organisations.

4.4 Summary

Variability challenges and used industrial practices are summarized in Table 5.

Our study indicates that increased product complexity can lead growing SMEs to the time consuming decision making negotiations. In addition, product complexity complicates the awareness of the status of core assets and product variants across the software organization not only in large companies but also in SMEs. This is in line with Thörn 2010 in (Current State and potential of variability management practices in software-intensive SMEs: results from a regional survey) who states that SMEs are often concerned that they are duplicating work due to the lack of communication.

5 Discussion

The interviewed development organisations were very small in Companies 1 to 4 (Table 2). The answers from all studied SMEs correspond with the following findings from previous work: *Lack of resources and budget to describe reuse assets and reuse processes, Small organisations are good at designing solutions for reuse, Competitive advantage via flexible products and services, Competitive advantage via specialized experts and face-to-face communication between them and Late requirement changes.* The SMEs' development projects, products and customer deliveries have focused for a long time on certain domains and market niches. This has allowed the development organisations to accumulate and make profitable use of knowledge and experience in the form of skilled personnel and product platforms. The skilled experts of the studied small organisations lean on face-to-face communication in developing and using reusable code assets. They are able to react expeditiously and appropriately to changes and customer needs. The code assets in the product platforms have an essential role in ensuring the quality of delivered products and the effectiveness of product development.

SMEs in previous work and in this study are lacking resources and budget to document information about reusable assets. However, this was not equally seen as a problem by the studied SMEs. The medium sized company 5 mentioned that it takes too much time to locate reusable platform components due to lacking documentation. On the other hand, reuse of previous features was not reported as a problem for Company 1 because its product family teams are small enough (less than ten in the team interviewed) that everyone knows which features have been built earlier.

The skilled experts enable most studied SMEs to use flexible and informal managerial practices in the turbulent business environment of the companies (Table 4). The use of agile development practices helped avoid unnecessary documentation work in three of this study's five SMEs.

Table 3 Differences and similarities between the findings from previous work and analysed SMEs related to reuse challenges

Finding from previous work	Answers from companies			Description
	Didn't agree	Partially agreed	Fully agreed	
Lack of resources and budget to describe reuse assets and reuse processes			C1, C2, C3, C5	Resource and time consuming documentation of product variants and core assets were reported as a variability challenge for three SMEs (C1, C2, C3 and C5).
There is a need for better documenting reusable assets		C1, C2, C3	C5	There is a challenge and a need to develop documentation practices for the used architectural style in Company 1 and for product feature selection in Company 2. Takes much time to locate reusable platform components in Company 5.
Little of requirements or design reuse	C4	C1, C2, C3	C5	All analysed SMEs used reusable features or requirements to describe offered customised products or services. Company 4 was actively reusing requirement descriptions, feature descriptions, interfaces and documentation. On the other hand, most of the companies reported needs to improve requirements, feature or design reuse.
Little traceability between the problem and solution spaces		C1, C2, C3	C5	Company 2 reported that oral communication is needed between selling people and product platform experts to trace between requirements and existing solutions in the product platform. The company has a need of developing a system that enables customers to select features from the product platform for the future release.
Small organisations are good at designing solutions for reuse			All	All of the analysed SMEs have a strong code reuse flow.
The observed benefits of systematic industrial reuse			C2	Only Company 2 mentioned that they have observed benefits of systematic industrial reuse. Company 2's product deliveries have been competitive especially in terms of fast deliveries, highly optimised features and free from defects due to the easy, fast, and automatized configuration process and the highly parameterised product base.
Proper product variability structures are seen useful			C1, C2	Company 1 has a competitive strategy to make customisations by setting service algorithm parameters without a need for changes to the basic product. Domain standards define and restrict variability structures for product, problem and solution spaces in Company 2. This allowed the company to construct and use a tool for partially automated generation of products and test data from the product platform.

Table 4 Differences and similarities between the findings from previous work and analysed SMEs related to variability management practices

Finding from previous work	Answers from companies			Description
	Didn't agree	Partially agreed	Fully agreed	
Competitive advantage <i>via</i> flexible products and services			All	All analysed SMEs offered customised products or services. Company 5 offered out of the box products as well. This study's results correspond with the finding.
Competitive advantage <i>via</i> specialized experts and face-to-face communication between them			All	The analysed companies were focused for a long time on a certain application domain and market niche.
Late requirement changes			All	One of the main development concerns of the companies of this study is the ability to respond to changes also during projects.
Flexible and informal managerial practices	C4	C3	C1, C2, C5	Three of the analysed companies (1, 2 and 5) use informal, collaborative and agile development conventions and face-to-face communication to manage development, to make decisions and to resolve problems. Company 3 uses more flexible practices in working with customers than in developing the company's own products. The use of agile development practices helped to avoid unnecessary documentation work in both Companies 2 and 5.
The number of supported product variants is low	C2, C3, C5	C1		Company 1 frequently delivers new product releases and delivers the newest version of the basic product to all customers. However, the parameter values of Company 1's service algorithm module are set for each customer. On the other hand, each of Company 2's 150 customers may have several product variants. This is also the case in Companies 3 and 5.
Customers are systems integrators	C1, C4, C5		C2, C3	

One company (C4) was actively reusing requirement descriptions, feature descriptions, interfaces and documentation. On the other hand, most of the companies reported problems in requirements or design reuse and documentation. This implicitly indicates little traceability between the problem and solution spaces.

Different product and customer types, delivery and business models seem to cause different answers related to some findings in Tables 3 and 4. The size of the development organisation does not seem to be the strongest factor in those findings. For example, only

Table 5 Variability management challenges and used industry practices in analysed SMEs

Challenge category	Challenge	Used industrial practice
Technical variability challenges		
Impact analysis	Product complexity is challenging to manage and difficult to understand for sales, marketing and customers (C1, C2, C4)	Prototypes helps when negotiating about requirements/changes (C3)
Product status management	Transparency to the core assets and user story/feature status (C5)	Frequent and open communication between sales agents and technical experts during sales negotiations (C2) Tool support for requirements management C2, C3, C5)
Decision making	Difficult decision making on changes due to unknown dependencies between existing system features (C2) Difficulties to measure the impact of the changes (C5)	Grouping team level feature estimations as a business case estimations that can be used when planning releases (C5) In B2B environment, negotiation based pricing principle for features, i.e., no fixed priced features (C2) Clear configuration strategies (C5) Continuous prioritization (C5)
Reuse	Reuse takes much time when the information of the reusable platform components has to be searched from source code (no documentation) (C5)	Use standard interfaces (C1) Use of parameters (C1, C2, C5) Building of modularity and reusability (C2)
Change implementation (architecture level)	Too time consuming change implementation (C5)	Re-engineering (C1) Partially automated generation of product releases (C2) A release tool for managing the versions of delivered releases (C2) To increase project profitability and reduce customer-specific variability, new features should be customisable to suit the needs of several customers (C2) Use of make files (C2)
Testing	Defect management of delivered products (C2, C4) No time and capacity for testing sufficiently delivered releases (C2) More test automation is needed (C2) More automation is needed to verify that deliveries are consistent with contracts (C2) A multiplicity of customers and different product versions (C2).	Test data generation from the system model (C2) Partially automated testing (C2) Open source components are not used due to defect, virus and other risks (C2) Changes by customers only at the level that hides dependencies on operating systems (C2) Keep history of the customer releases in order to check how many/which customers have this defect (C2)
Tool support	Lack of tool support makes it difficult to manage product variants (C2, C3)	Test automation (C2, C5) Use of parameters (C1, C2, C5)

Table 5 (continued)

Challenge category	Challenge	Used industrial practice
People Variability Challenges		Partially automated generation of products using a configuration tool (C2)
		Tool support for customer release management (C2, C5)
Resourcing	Not enough resources for documentation (C2, C3 C5)	Documentation is automatically generated as a part of the generation of product releases (C2)
	Not enough resources, same developer is working in too many projects at the same time (C2, C3, C5)	Define responsibilities in detailed in the maintenance contract (C2)
Business models	Uncertainty of the revenue model (C2)	Efficient customer collaboration (C5)
	It is challenging to move to the new markets and deal with the vertical markets (C5)	

Company 2 described observed benefits of systematic industrial reuse. Only this of the studied companies has a partially automated product configuration process. The company uses a script language, a configuration tool, make files and configuration parameters for the partially automated generation of release delivery packages to customers.

Companies 1 and 2 reported useful domain-specific product variability structures. Company 1 makes large customisations by setting service algorithm parameters without a need for changes to the basic product. Company 2's product variability structures are based on domain standards and have been implemented using a script language, make files and configuration parameters which allow partially automated generation of products from the product platform.

Two of the analysed SMEs (Companies 2 and 3) are doing consulting work in addition to customising products on a per-contract basis from the product base for integrator customers. The different business models support each other. This result is not the same as the finding from Thörn's (2010) survey.

The finding "The number of supported product variants is low" is fully agreed by no studied company. Maintenance costs tend to grow when the number of supported product variants grows. Therefore, for example Companies 1 and 2 mentioned the need to limit the growth of the number of supported product variants. However, it is often difficult to do when simultaneously gaining competitive advantage *via* flexible products and late requirements changes and customisations. For example, Companies 2 and 3 have to keep a history of the customers and the deployed releases at their premises because a different combination of assets in the reuse platform is delivered to each system integrator customer. On the other hand, for keeping the maintenance costs of supported product variants low, Company 1 delivers the newest version of the basic product to all customers. The basic product is kept suitable for all customers and most customizations are made by setting parameter values of service algorithms without changes to the basic product.

Results suggests that the product complexity can lead SMEs to a situation in which it is difficult to manage the continuous flow of defects originating from different customers and end users. Contrary to Thörn's (2010) argument that SMEs would be less interested in variability management tools, the analysed medium size of companies expect that tool

support can boost their productivity when managing complex products and many variants. In fact, many tools already exist for managing variability (e.g. Gears, and pure::variants) (Chen and Ali Babar 2010). However, the majority of the analysed companies used only parameterisation (Svahnberg et al. 2005) to realise variability.

Automation is particularly important when managing product requirements, user stories, design and testing artefacts (Bosch et al. 2001). In the analysed SMEs, a high level of automation in design, release management and testing are or become a key factor for market success.

In our analysis, reuse was reported as an important approach to survive in the dynamic situations in all case companies. In fact, it was identified as a way to assure fast product implementation to the different customers. Although, the findings suggest that reuse in general is a good practice, one of our case companies reported it to be an expensive approach that dramatically increases complexity. It was mentioned, for instance, in a medium size company that reusing product artefacts can easily be too time consuming without sufficient documentation of the previously developed features. However, our study shows that creating documentation or models for reuse is not often possible for small companies due to resource constraints.

Thörn (2010) states that the SME companies are often concerned with work duplication and communication problems within the development organisation due to the lack of variability structures. Our case companies, however, also reported that the increased complexity and product feature dependencies are difficult to understand especially for sales, marketing and customers who often do not have the needed technical background to understand the product and product interface complexity.

6 Conclusions & Future Work

The importance of the use of research solutions for more efficient variability management is increasing in software intensive companies. The purpose of this study was to identify and analyse challenges that SMEs are currently facing, and practices they are applying, for managing variability. In this paper, it was discussed that the increased product complexity can lead a growing SME to the time consuming decision making and lack of knowledge about and the status of core assets and product variants across the organization. Defect management and documentation activities were reported as a challenge in several SMEs. Furthermore, it was stated to be difficult to decide whether the resources are divided into the parallel release development or maintenance activities. Better knowledge of business model and variability management practices would help all companies, but especially SMEs to find better ways to tackle with the increasing amount of variability problems. Additionally it was found that in the analysed SMEs, a high level of automation and tool support in design, release management and testing are or are becoming as a key factor for their market success. From the reuse, our case companies reported it to be a good but an expensive approach that can dramatically increase complexity of the management in organizations.

Many findings of this study correspond with the findings from previous work related to variability management challenges and practices in small and very small development organisations. The small organisations in this study and in previous work are lacking resources and budget to document information about reusable assets. They are good at designing solutions for reuse. They gain competitive advantage *via* flexible products and services, specialized experts and face-to-face communication between them. Their skilled experts enable to use flexible and informal managerial practices in the turbulent business environment of the companies. Customer satisfaction is often determined by the fulfilment of specific requirements that may change during the project.

Despite the addressed limitations of this study, we hope that the results will provide useful information for both research communities and industry on their way towards the

optimized software variability management. In the future our goal is to focus the research more on the variability from the business and product transparency perspective. Another purpose is to use results of this study to make a survey that validates the organizational challenges and used industrial practices in more large set of companies and larger amount of company representatives.

Acknowledgments The research leading to these results has received funding from the ARTEMIS Joint Undertaking under grant agreement n° 295397 and from Tekes (the Finnish Funding Agency for Technology and Innovation) and IWT.

References

- Ali Babar M, Chen L, Shull F (2010) Managing variability in software product lines. *IEEE Softw* 27(3):89–94
- Asikainen T, Mannisto T, Soininen T (2007) Kumbang: A domain ontology for modelling variability in software product families. *Adv Eng Inform* 21(1):23–40
- Bachmann F, Clements PC (2005) Variability in software product lines. SEI CMU, Pittsburgh, Technical Report CMU/SEI-2005-TR-012
- Bosch J (2006) The challenges of broadening the scope of software product families the challenges of brooding the cope of software product families. *Commun ACM* 49(12):41–44
- Bosch J, Florijn G, Greefhorst D, Kuusela J, Obbink H, Pohl K (2001) Variability issues in software product lines. In: van der Linden F (ed) *Fourth International Workshop on Product Family Engineering (PFE-4)*. Springer-Verlag, Bilbao, pp 13–21
- Chen L, Ali Babar M (2010) Variability management in software product lines: an investigation of contemporary industrial challenges. *Proceeding SPLC'10 Proceedings of the 14th international conference on Software product lines*
- Chen L, Ali Babar M (2011) A systematic review of evaluation of variability management approaches in software product lines. *Inf Softw Technol* 53: 344–362
- Clements P, Northrop L (2001) *Software product lines*. Addison-Wesley, New York
- Codenie W, Gonzalez-Deleito N, Deleu J, Blagojevic V, Kuvaja P, Similä J (2010) Managing flexibility and variability: a road to competitive advantage. In: Kang K, Sugumaran V, Park S (eds) *Applied software product line engineering*. CRC Press, New York, pp 269–313
- Eisenhardt KM (1989) Building theories from case study research. *Acad Manag Rev* 14(4):532–550
- Eisenhardt KM, Graebner ME (2007) Theory building from cases: opportunities and challenges. *Acad Manag J* 50(1):25–32
- Fillis I (2001) Small firm internationalisation: an investigative survey and future research directions. *Manag Decis* 39(9):767–783
- Galster M, Avgeriou P (2011) Handling variability in software architecture: problems and implications. *Proceedings of the ninth working IEEE/IFIP Conference on Software Architecture* pp 171–180
- Groves L, Nickson R, Reeve G, Reeves S, Utting M (2000) A survey of software development practices in New Zealand software industry. *Aswec (Australian Software Engineering Conference) 00*, pp 189–202
- Gurp J, Bosch J, Svahnberg M (2001) On the notion of variability in software product lines. *Proc Work IEEE/IFIP Conf Softw Archit* 2001:45–55
- Hallsteinsen S, Hinchey M, Park S, Schmid K (2008) *Dyn Softw Prod Lines Comput* 2008:93–95
- Halmans G, Pohl K (2003) Communicating the variability of a software-product family to customers. *Softw Syst Model* 2(1):15–36
- Info-Tech (2004) Stage 7 >build a strategic decision making framework. In: *Strategic IT planning and governance, info-tech research group* (www.infotech.com)
- Jaeeoon L, Kotonya G, Robinson D (2012) Engineering service-based dynamic software product lines. *IEEE Comput* 45(10):49–54
- Kamsties E, Hörmann K, Schlich M (1998) Requirements engineering in small and medium enterprises. *Requir Eng* 3(2):84–90
- Karlström D, Runeson P (2006) Integrating agile software development into stage-gate managed product development. *Empir Softw Eng* 11(2):203–225
- Kim KS, Knotts TL, Jones SC (2008) Characterizing viability of small manufacturing enterprises (SME) in the market. *Expert Syst Appl* 34(2008):128–134

- Kratochvil M, Carson C (2005) Growing modular: mass customization of complex products, services and software. Springer
- Krueger C (2001) Variation management for software production lines. In: Chastek G (Ed) SPLC2 2002, LNCS 2379, pp 37–48
- Krueger C (2002) Eliminating the adoption barrier. *IEEE Softw* 19(4):29–31
- Laporte CY, Alexandre S, O'Connor RV (2008) A software engineering lifecycle standard for very small enterprises. Proceedings of the 15th European Software Process Improvement Conference, EuroSPI 2008. EuroSPI 2008, Dublin, pp 129–141. doi:[10.1007/978-3-540-85936-9_12](https://doi.org/10.1007/978-3-540-85936-9_12)
- Manolova T, Brush C, Edelman L, Greene P (2002) Internationalization of small firms: personal factors revisited. *Int Small Bus J* 20(1):9–31
- Metha A, Heineman GT (2002) Evolving legacy system features into fine grained components. *ICSE* 2002:417–427
- Mohagheghi P, Conradi R (2007) Quality, productivity and economic benefits of software reuse: a review of industrial studies. *Empir Softw Eng* 12(5):471–516
- Mtshwe B (2005) The entrepreneurial firm internationalization process in the Southern African context: a comparative approach. *Int J Entrep Behav Res* 11(5):358–377
- Nikula U, Sajaniemi J, Kälviäinen H (2000) A state-of-the-practice survey on requirements engineering in small- and medium-sized enterprises, research report 1. Telecom Business Research Center, Lappeenranta University of Technology, Lappeenranta, 20 p
- Northrop L (2010) Maximizing the investment from your software product portfolio. Software Engineering Institute. Available at <http://www.sei.cmu.edu/library>
- Patton MQ (2002) Qualitative research & evaluation methods. Sage Publications, London
- Pohl K, Böckle G, van der Linden F (2005) Software product-line engineering—foundations. Principles and Techniques, Springer
- Rech J, Bogner C, Haas V (2007) Using wikis to tackle reuse in software projects. *IEEE Softw* 24(6):99–104
- Reinhartz-Berger I, Sturm A (2009) Utilizing domain models for application design and validation. *Inf Softw Technol* 51:1275–1289
- Runeson P, Höst M (2009) Guidelines for conducting and reporting case studies in software engineering. *Empir Softw Eng* 14:131–164
- Stallinger F, Neumann R, Schossleitner R, Stephan Kriener S (2011) Migrating towards evolving software product lines: challenges of an SME in a core customer-driven industrial systems engineering context. Proceedings of PLEASE'11, May 22–23, 2011, Waikiki, Honolulu, pp. 20–24
- Svahnberg M, van Gurp J, Bosch A (2005) A taxonomy of variability realization techniques. *Softw Pract Experience* 35(8):705–754
- Thörn C (2010) Current state and potential of variability management practices in software-intensive SMEs: results from a regional survey. *Inf Softw Technol* 52:411–421
- van der Linden FJ, Schmid K, Rommes E (2007) Software product lines in action. Springer, Berlin, 333 p
- Verlage M, Kiesgen T (2005) Five years of product engineering in small company, *ICSE*, pp 534–543
- Yin RK (2002) Case study research design and methods, 3rd edn. Sage Publications, Thousand Oaks



Tuomas Ihme is a Senior Scientist at VTT, the Technical Research Centre of Finland. His professional experience involves several years in the industry as a project manager as well as more than 20 years in research of embedded software and management of industrial development projects and national joint research projects. His areas of expertise are architectural design methods, modelling tools and software architectures in

component software, product lines and agile software development. He has authored more than thirty scientific publications.



Minna Pikkarainen is a Principal Scientist at the VTT Technical Research Centre of Finland. She has worked in several industrial-driven research projects and project preparations, collaborating closely with software companies across Europe. Minna's research has been published in 25+ journal and conference papers in the forums like ICSE, ICIS and Empirical Software Engineering Journal. Minna has been member of Lero, The Irish Software Engineering Research Centre and Sirris, The Collective Centre of the Belgian Technology Industry. Currently Minna Pikkarainen is working as a business developer in Institute Mines Telecom, Paris and EIT ICT Labs, Helsinki. Recently, her research has been focused on the areas of agile development, software innovation and variability management.



Susanna Teppola has worked as a Research Scientist at VTT since 2000. Susanna has over fourteen years' experience in ICT, her current research interests laying in the area of product and service management, software development processes and practices and model driven development. In these areas Susanna has participated in many industrial and industry-driven research projects and project preparations both at national and international level.



Jukka Kääriäinen works as a Senior Scientist at VTT ICT cluster (Service development and management - team). He has received Ph.D degree 2011 in Computer Science. He has over 10 years of experience with configuration management and lifecycle management. He has been involved in various European ITEA, ITEA2 and Artemis research projects.



Olivier Biot is a Technology and Innovation Advisor at Sirris, The Collective Centre of the Belgian Technological Industry. He helps companies in various domains with technology innovation, typically where software engineering and ICT meet other disciplines. He started in the telecommunications and mobile Internet industry. His very broad technology affinity brings him in contact with many high-tech companies.